

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-a56929604f77b9d41.jsonl`
- SHA-256: `d55f62d5e0e3c6b66741ecfff5fa5107f08a7ad21cf188d103a5fb46bc1b0a67`
- Source modified: `2026-06-07T09:11:42+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`
```

Transcript

1. user (2026-06-07T09:10:30.786Z)

Read these files in C:/Users/avidu/Projects/badminton-highlight-indexer and produce a DENSE, anchor-rich CODE_MAP chunk for the "Infrastructure & reporting (observability)" subsystem. Use the format: bullet per file/symbol with anchors written as @path::symbol where @path is the repo-relative path EXACTLY as listed (the current layered layout). Capture: key classes/functions/consts; data contracts (mark with \$); registries / plugin extension points (mark with ?); and footguns/gotchas (mark with ?). Be precise ? only list symbols that actually exist in the files. If a path is a directory (e.g. tests/), enumerate its files and map each test to the subsystem it covers.

Files/paths: backend/infrastructure/database.py, backend/reporting/__init__.py, backend/reporting/harvest.py, backend/reporting/spec.yaml

2. assistant (2026-06-07T09:10:33.328Z)

I'll read all four files to produce the CODE_MAP chunk.

3. user (2026-06-07T09:10:34.236Z)

```
1      import sqlite3
2      import json
3      import os
4      import logging
5      from typing import List, Dict, Any, Optional
6
7      logger = logging.getLogger(__name__)
8
9      class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         def _get_connection(self):
15             conn = sqlite3.connect(self.db_path)
16             conn.row_factory = sqlite3.Row
17             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
18             conn.execute("PRAGMA foreign_keys = ON")
19             return conn
20
21         def _init_db(self):
22             """Initializes tables for videos, play_segments, and events."""
23             with self._get_connection() as conn:
24                 cursor = conn.cursor()
25
26                 # Videos Table
27                 cursor.execute("""
```

```

28         CREATE TABLE IF NOT EXISTS videos (
29             id TEXT PRIMARY KEY,
30             filepath TEXT NOT NULL,
31             processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32             status TEXT NOT NULL,
33             validation_results TEXT
34         )
35     """)
36
37     # Play Segments Table (Extensible metadata JSON)
38     cursor.execute("""
39         CREATE TABLE IF NOT EXISTS play_segments (
40             id INTEGER PRIMARY KEY AUTOINCREMENT,
41             video_id TEXT NOT NULL,
42             start_time REAL NOT NULL,
43             end_time REAL NOT NULL,
44             duration REAL NOT NULL,
45             server TEXT,
46             receiver TEXT,
47             metadata TEXT,
48             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49         )
50     """)
51
52     # Events Table
53     cursor.execute("""
54         CREATE TABLE IF NOT EXISTS events (
55             id INTEGER PRIMARY KEY AUTOINCREMENT,
56             segment_id INTEGER NOT NULL,
57             timestamp REAL NOT NULL,
58             type TEXT NOT NULL,
59             player TEXT,
60             details TEXT,
61             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
62             CASCADE
63         )
64     """)
65
66     # Versioned migrations live here. PRAGMA user_version is the schema
67     # contract ? bump it for every change and add an ordered `if version < N`
68     # block below. Tests assert the expected version, so accidental drift fails
69
70     version = cursor.execute("PRAGMA user_version").fetchone()[0]
71
72     if version < 1:
73         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
74         # Common fields are columns; detector-specific metrics go in `stats`
75         # so a new segmenter reuses this table by setting its own
76         cursor.execute("""
77             CREATE TABLE IF NOT EXISTS candidate_segments (
78                 id INTEGER PRIMARY KEY AUTOINCREMENT,
79                 video_id TEXT NOT NULL,
80                 detector TEXT NOT NULL,
81                 chunk_index INTEGER,
82                 start_time REAL NOT NULL,
83                 end_time REAL NOT NULL,
84                 duration REAL NOT NULL,
85                 confidence REAL,
86                 stats TEXT,
87                 detection_params TEXT,
88                 confirmed_segment_id INTEGER,
89                 human_label TEXT,
90                 notes TEXT,

```

```

89             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
90             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
91             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
92         )
93     """)
94     cursor.execute("""
95         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
96         ON candidate_segments(video_id, detector)
97     """)
98     cursor.execute("PRAGMA user_version = 1")
99     # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 2
100
101     conn.commit()
102
103     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
104         try:
105             with self._get_connection() as conn:
106                 conn.cursor().execute(
107                     "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
108                     (video_id, filepath, status, json.dumps(validation_results))
109                 )
110                 conn.commit()
111                 return True
112             except Exception as e:
113                 logger.error(f"Failed to add video: {e}")
114                 return False
115
116     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
117         try:
118             with self._get_connection() as conn:
119                 cursor = conn.cursor()
120                 cursor.execute(
121                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
122                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
123                 )
124                 conn.commit()
125                 return cursor.lastrowid
126             except Exception as e:
127                 logger.error(f"Failed to add play segment: {e}")
128                 return None
129
130     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
131         try:
132             with self._get_connection() as conn:
133                 conn.cursor().execute(
134                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
135                     (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
136                 )
137                 conn.commit()
138                 return True
139             except Exception as e:
140                 logger.error(f"Failed to add event: {e}")
141                 return False
142

```

```

143     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
144                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
145                               stats: Optional[Dict[str, Any]] = None,
146                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
147         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
148         detector-specific dicts stored as JSON. Returns the new candidate id, or
149         None."""
150         try:
151             with self._get_connection() as conn:
152                 cursor = conn.cursor()
153                 cursor.execute(
154                     """INSERT INTO candidate_segments
155                     (video_id, detector, chunk_index, start_time, end_time, duration,
156                     confidence, stats, detection_params)
157                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
158                     (video_id, detector, chunk_index, start_time, end_time, end_time -
159                     start_time,
160                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
161                     {})))
162                 )
163                 conn.commit()
164                 return cursor.lastrowid
165         except Exception as e:
166             logger.error(f"Failed to add candidate segment: {e}")
167             return None
168
169     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
170         """Record that a candidate detection was confirmed as a given play_segment."""
171         try:
172             with self._get_connection() as conn:
173                 conn.cursor().execute(
174                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
175                     ?",
176                     (segment_id, candidate_id)
177                 )
178                 conn.commit()
179             return True
180         except Exception as e:
181             logger.error(f"Failed to link candidate {candidate_id} to segment
182             {segment_id}: {e}")
183             return False
184
185     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
186                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
187         """Fetch candidate detections for the annotation / fine-tuning workflow.
188         `stats` and `detection_params` are deserialized from JSON."""
189         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
190         params: List[Any] = [video_id]
191
192         if detector:
193             query += " AND detector = ?"
194             params.append(detector)
195         if only_unlabeled:
196             query += " AND human_label IS NULL"
197
198         query += " ORDER BY start_time"
199
200         candidates = []
201         with self._get_connection() as conn:
202             rows = conn.cursor().execute(query, params).fetchall()
203             for row in rows:
204                 d = dict(row)

```

```

200         d["stats"] = json.loads(d["stats"] or "{}")
201         d["detection_params"] = json.loads(d["detection_params"] or "{}")
202         candidates.append(d)
203     return candidates
204
205     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
206         with self._get_connection() as conn:
207             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
208 (video_id,)).fetchone()
209             if row:
210                 d = dict(row)
211                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
212                 return d
213             return None
214
215     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
216 List[Dict[str, Any]]:
217         """
218         Dynamically queries play segments based on filters:
219         - duration_min: float
220         - server: string
221         - receiver: string
222         - event_type: string (e.g. smash, foul)
223         """
224         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
225         params = [video_id]
226
227         if filters.get("duration_min"):
228             query += " AND r.duration >= ?"
229             params.append(filters["duration_min"])
230
231         if filters.get("server"):
232             query += " AND r.server = ?"
233             params.append(filters["server"])
234
235         if filters.get("receiver"):
236             query += " AND r.receiver = ?"
237             params.append(filters["receiver"])
238
239         if filters.get("event_type"):
240             # Check if there is an event of a specific type inside this segment
241             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
242 e.type = ?)"
243             params.append(filters["event_type"])
244
245         segments_list = []
246         with self._get_connection() as conn:
247             cursor = conn.cursor()
248             rows = cursor.execute(query, params).fetchall()
249
250             for row in rows:
251                 r_dict = dict(row)
252                 r_id = r_dict["id"]
253                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
254
255                 # Fetch associated events
256                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
257 (r_id,)).fetchall()
258                 r_dict["events"] = []
259                 for er in event_rows:
260                     e_dict = dict(er)
261                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
262                     r_dict["events"].append(e_dict)
263
264                 segments_list.append(r_dict)

```

```

261
262         return segments_list
263
264     def clear_video_data(self, video_id: str):
265         with self._get_connection() as conn:
266             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
267             conn.commit()
268

```

4. user (2026-06-07T09:10:34.727Z)

```

1     """Per-video observability reports for the indexer.
2
3     `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4     no-op and the pipeline runs exactly as before. The single entrypoint
5     `emit_indexer_report(...)` is designed to be called from a `finally` block and
6     NEVER raises ? emitting a report must not be able to break a processing run.
7     """
8     from __future__ import annotations
9
10    import os
11    from typing import Any, Dict, List, Optional
12
13    try: # soft dependency ? never hard-fail the pipeline on a reporting import
14        import obsreport # noqa: F401
15        from . import harvest
16
17        OBSREPORT_AVAILABLE = True
18    except Exception: # pragma: no cover - exercised only when obsreport is absent
19        OBSREPORT_AVAILABLE = False
20
21
22    def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
23        return bool((config or {}).get("report", {}).get("enabled", True))
24
25
26    def emit_indexer_report(
27        *,
28        db,
29        video_id: str,
30        video_path: Optional[str],
31        config: Optional[Dict[str, Any]],
32        val_results: List[Any],
33        val_context: Optional[Dict[str, Any]] = None,
34        segmenter_requested: Optional[str] = None,
35        segmenter_actual: Optional[str] = None,
36        was_reingest: bool = False,
37        segmentation_ran: bool = True,
38    ) -> Optional[Dict[str, str]]:
39        """Harvest + write the report. Returns the written file paths, or None if
40        reporting is unavailable/disabled/failed. Never raises."""
41        config = config or {}
42        # Accept either a plain dict or a typed config object (e.g.
43        backend.config.AppConfig).
44        # Normalize to a plain nested dict so harvest's dict access works uniformly.
45        if hasattr(config, "model_dump"):
46            try:
47                config = config.model_dump(mode="json")
48            except Exception:
49                config = {}
50        if not OBSREPORT_AVAILABLE or not report_enabled(config):
51            return None
52        try:
53            play_segments = db.query_play_segments(video_id, {}) if db is not None else []
54            candidates = db.query_candidate_segments(video_id) if db is not None else []

```

```

54         rec = harvest.record_run(
55             video_id=video_id,
56             video_path=video_path,
57             config=config,
58             val_results=val_results,
59             val_context=val_context,
60             play_segments=play_segments,
61             candidates=candidates,
62             segmenter_requested=segmenter_requested,
63             segmenter_actual=segmenter_actual,
64             was_reingest=was_reingest,
65             segmentation_ran=segmentation_ran,
66         )
67         output_dir = (config.get("report") or {}).get("output_dir")
68         res = rec.write(output_dir=output_dir)
69         return res.paths or None
70     except Exception: # pragma: no cover - defensive; reporting must never break a run
71         return None
72

```

5. user (2026-06-07T09:10:35.648Z)

```

1     """Build an observability report for one indexer run.
2
3     Pure-ish harvest: given the validation results/context, the persisted segments,
4     and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5     *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6     so this module only records facts faithfully ? it does not decide warnings.
7
8     Imported only after `backend.reporting` confirms obsreport is installed, so it may
9     import obsreport freely.
10    """
11    from __future__ import annotations
12
13    import os
14    from pathlib import Path
15    from typing import Any, Dict, List, Optional
16
17    import obsreport
18
19    _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
20    _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
21
22
23    def load_spec() -> "obsreport.ReportSpec":
24        return obsreport.load_spec(_SPEC_PATH)
25
26
27    # --- media metadata -----
28    def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
29        """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
30        distinguishes a real probe from the absence of one (the fallback footgun)."""
31        meta: Dict[str, Any] = {"fps_source": "unknown"}
32        if video_path and os.path.exists(video_path):
33            try:
34                meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
35            except OSError:
36                pass
37
38        ffmata = (val_context or {}).get("ffprobe_meta") or {}
39        streams = ffmata.get("streams") or []
40        fmt = ffmata.get("format") or {}
41        if streams:
42            s = streams[0]

```

```

43         w, h = s.get("width"), s.get("height")
44         meta["width"], meta["height"] = w, h
45         if w and h:
46             meta["resolution"] = f"{w}x{h}"
47         fps_str = s.get("r_frame_rate", "0/1")
48         try:
49             num, den = (int(x) for x in fps_str.split("/"))
50             if den:
51                 meta["fps"] = round(num / den, 2)
52                 meta["fps_source"] = "probed"
53         except (ValueError, ZeroDivisionError):
54             pass
55         if s.get("codec_name"):
56             meta["container"] = fmt.get("format_name") or s.get("codec_name")
57     if fmt.get("duration"):
58         try:
59             meta["duration_s"] = round(float(fmt["duration"]), 1)
60         except (ValueError, TypeError):
61             pass
62     if fmt.get("format_name"):
63         meta["container"] = fmt.get("format_name")
64     return meta
65
66
67 # --- stage builders -----
68 def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
69     with rec.stage("validation") as st:
70         total = len(val_results)
71         passed = sum(1 for r in val_results if getattr(r, "passed", False))
72         st.add_metric("checks_passed", passed)
73         st.add_metric("checks_total", total)
74         failures = [r for r in val_results if not getattr(r, "passed", True)]
75         for r in failures:
76             st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r, 'message', '')}")
77         st.status = "error" if failures else ("ok" if total else "not_run")
78
79
80 def _segmentation_stage(
81     rec: "obsreport.RunRecorder",
82     play_segments: List[Dict[str, Any]],
83     candidates: List[Dict[str, Any]],
84     segmenter_requested: Optional[str],
85     segmenter_actual: Optional[str],
86     config_default: Optional[str],
87     was_reingest: bool,
88     ran: bool,
89 ) -> None:
90     with rec.stage("segmentation") as st:
91         if not ran:
92             st.status = "not_run"
93         seg_count = len(play_segments)
94         total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
95         detector_tag = None
96         if play_segments:
97             detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
98         st.add_metric("candidate_windows", len(candidates))
99         st.add_metric("segment_count", seg_count, audience="both")
100        st.add_metric("total_play_s", total_play, unit="s")
101        # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
102        # pick it, and it differs from config's default (the genuine surprise case).
103        # An explicit --segmenter override or a halted run is not a divergence.
104        matches_default = True
105        if ran and segmenter_actual and config_default:
106            matches_default = (segmenter_actual == config_default)

```



```

107         st.details.update({
108             "segmenter_requested": segmenter_requested,
109             "segmenter_actual": segmenter_actual,
110             "config_default": config_default,
111             "segmenter_explicitly_requested": segmenter_requested is not None,
112             "matches_config_default": matches_default,
113             "was_reingest": was_reingest,
114             "detector": detector_tag,
115         })
116         # Surface metadata trustworthiness at the data level (not just in a warning):
117         # the motion baseline fabricates per-segment metadata; everything else measures
118         it.
119         if ran and segmenter_actual:
120             st.details["metadata_source"] = (
121                 "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
122                 else "measured (detector/AI)"
123             )
124
125     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
126                 play_segments: List[Dict[str, Any]], ran: bool) -> None:
127         """Only meaningful for AI-based segmenters. Records whether the provider key was
128         present ? the signal the silent_provider_noop rule keys on."""
129         ai_based = segmenter_actual in _AI_SEGMENTERS
130         provider = (config.get("ai_provider") or "gemini")
131         key_env = f"{provider.upper()}_API_KEY"
132         with rec.stage("ai_handoff") as st:
133             if not ran or not ai_based:
134                 st.status = "not_run" if not ran else "skipped"
135             st.details.update({
136                 "ai_based": ai_based,
137                 "provider": provider if ai_based else None,
138                 "model": config.get("ai_model") if ai_based else None,
139                 "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
140             })
141             if ai_based:
142                 st.add_metric("confirmed_segments", len(play_segments))
143
144
145     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
146                   video_meta: Dict[str, Any]) -> None:
147         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
148         coverage = None
149         dur = video_meta.get("duration_s")
150         if dur:
151             coverage = round(100.0 * total / dur, 1)
152         notes = []
153         if not play_segments:
154             notes.append("No highlights were produced for this video.")
155         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
156                           coverage_pct=coverage, notes=notes)
157
158
159     # --- top-level -----
160     def record_run(
161         *,
162         video_id: str,
163         video_path: Optional[str],
164         config: Dict[str, Any],
165         val_results: List[Any],
166         val_context: Optional[Dict[str, Any]],
167         play_segments: List[Dict[str, Any]],
168         candidates: List[Dict[str, Any]],
169         segmenter_requested: Optional[str] = None,

```

```

170     segmenter_actual: Optional[str] = None,
171     was_reingest: bool = False,
172     segmentation_ran: bool = True,
173     tool_version: Optional[str] = None,
174     spec: Optional["obsreport.ReportSpec"] = None,
175     **recorder_kwargs: Any,
176 ) -> "obsreport.RunRecorder":
177     """Assemble a finalized recorder for an indexer run (does NOT write)."""
178     rec = obsreport.RunRecorder(
179         "badminton-highlight-indexer",
180         tool_version=tool_version,
181         video_id=video_id,
182         video_id_kind="file_md5",
183         video_path=video_path,
184         spec=spec if spec is not None else load_spec(),
185         **recorder_kwargs,
186     )
187     vm = video_meta_from_context(val_context, video_path)
188     # If we never probed an fps but segmentation still ran, the segmenter must have
189     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
190     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?
(fallback)".
191     if vm.get("fps_source") == "unknown" and segmentation_ran:
192         vm["fps_source"] = "fallback"
193         if vm.get("fps") is None:
194             vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
195     rec.set_video_meta(**vm)
196
197     _validation_stage(rec, val_results)
198     config_default = (config.get("indexing") or {}).get("default_segmenter")
199     _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
200                        segmenter_actual, config_default, was_reingest,
segmentation_ran)
201     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran)
202     _highlights(rec, play_segments, vm)
203
204     rec.finalize()
205     return rec
206

```

6. user (2026-06-07T09:10:35.757Z)

```

1     # Observability report spec for the badminton-highlight-indexer.
2     # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3     # library; this file is the indexer-specific layout + footgun rules. Every
4     # actionable rule cites a README/FAQ section so a reader can self-debug.
5     name: indexer
6     title: Badminton Highlight Indexer ? Video Report
7
8     # Customer here = whoever receives the highlight reel.
9     customer_verdict:
10         pass: "? Your highlights are ready."
11         pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12         fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14     sections:
15         - { id: run,          title: "Run",          kind: identity,  audience: both }
16         - { id: video,       title: "Video",        kind: video_meta, audience: both }
17         - { id: highlights,  title: "Highlights",    kind: highlights, audience: customer
}
18         - { id: cust_notes,  title: "Things to know", kind: flags,      audience: customer
}
19         - { id: stages,     title: "Processing stages", kind: stages,     audience:

```

```

technical }
20   - { id: findings,    title: "Findings & warnings", kind: flags,      audience:
technical }
21   - { id: verdict,     title: "Verdict",                kind: verdict,    audience: both }
22
23   rules:
24     # The classic silent failure: 0 segments because the AI provider no-op'd on a
25     # missing API key ? NOT because the video has no rallies.
26     - code: silent_provider_noop
27       severity: error
28       when:
29         - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
30         - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
31       message: >-
32         0 segments AND no AI provider API key was set. The segmenter (see Processing
33         stages > segmentation > segmenter_actual) hands its candidate windows to the AI
34         provider, which silently did nothing without a key ? so this is a MISSING
35         CREDENTIAL, not 'the video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a
36         .env file in the project root and re-run (see README#Q7).
37       faq_ref: "README#Q7"
38       customer_message: >-
39         We couldn't generate highlights for this video due to a configuration issue
40         on our side. Please contact support.
41
42     # API key WAS present but an AI segmenter still produced nothing ? likely a
43     # provider error / quota / WSL healthcheck issue rather than a truly empty video.
44     - code: ai_empty_vs_no_rallies
45       severity: warn
46       when:
47         - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
48         - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
49         - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
50       message: >-
51         An AI-based segmenter returned 0 segments even though the API key is set.
52         Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
53         failed WSL healthcheck ? these all return [] silently. Re-run with -u for
54         unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
55       faq_ref: "README#Q7"
56
57     # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
58     - code: fps_fallback_used
59       severity: warn
60       when:
61         - { path: "video_meta.fps_source", op: eq, value: fallback }
62       message: >-
63         fps was NOT probed from the file ? a fallback default (shown as the fps value)
64         was used. Because rally timing and motion thresholds scale with fps, boundaries
65         and motion sensitivity may be miscalibrated. This usually means validation didn't
66         run (it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real
67         fps with ffprobe and re-run.
68       faq_ref: "README#Q14"
69
70     # The motion baseline fabricates its per-segment metadata/events ? they are not
71     # measured. Warn so nobody trusts them as ground truth.
72     - code: synthetic_motion_metadata
73       severity: warn
74       when:
75         - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
76       message: >-
77         The 'motion' segmenter was used: its per-segment metadata (intensity, speed,
78         events, server/receiver) are PLACEHOLDER values (unmeasured) ? fabricated by the
79         heuristic, NOT derived from the video ? so a PASS with segments here does not mean
80         the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
81       faq_ref: "README#Q10"
82

```

```

83      # The segmenter that ran differs from config's default -> you may be looking at a
84      # different detector than expected (config/setup/CLI defaults are known to diverge).
85      - code: segmenter_divergence
86      severity: info
87      when:
88        - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
false }
89        - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
value: false }
90      message: >-
91        The segmenter that ran differs from config.json's default_segmenter, and no
92        explicit --segmenter was given ? so a fallback default kicked in. The indexer's
93        defaults are known to diverge across config.json / setup.py / the CLI; confirm
94        you ran the detector you meant.
95      faq_ref: "README#Q7"
96
97      # Re-processing an already-indexed video replaced its prior segments (INSERT OR
98      # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
99      - code: reingest_replaced_prior
100     severity: info
101     when:
102       - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
103     message: >-
104       This video was already in the database; re-processing REPLACED its previous
105       segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
106       when you intend to re-run; just know the earlier results are gone.
107     faq_ref: "README#Q11"
108
109     # A validation check failed -> segmentation was halted. Point the reader at the
110     # report's Validation section + the per-check FAQ.
111     - code: validation_failed
112     severity: error
113     when:
114       - { path: "stages.validation.status", op: eq, value: error }
115     message: >-
116       One or more validation checks failed, so indexing was halted. See the
117       Processing stages > validation block above: each failed check states the
118       reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
119     faq_ref: "README#Q13"
120     customer_message: >-
121       This video couldn't be processed because it didn't pass our quality checks
122       (for example: too blurry, too low resolution, or not a badminton court).
123

```

7. assistant (2026-06-07T09:11:41.975Z)

```

@backend/infrastructure/database.py
- `Database` class ? single SQLite gateway; opens a fresh connection per call.
@backend/infrastructure/database.py::Database
- `Database.__init__(db_path)` ? stores `db_path`, calls `_init_db()` on construction.
@backend/infrastructure/database.py::Database.__init__
- `Database._get_connection()` ? `sqlite3.connect`, sets `row_factory = sqlite3.Row`, runs
`PRAGMA foreign_keys = ON`. ? FK enforcement is per-connection in SQLite and OFF by default ?
every connection MUST re-run this PRAGMA or `ON DELETE CASCADE`/`SET NULL` silently won't fire.
@backend/infrastructure/database.py::Database._get_connection
- `Database._init_db()` ? creates 3 base tables + runs versioned migration block. ? Migration
contract: `PRAGMA user_version` is the schema version; bump it for every change and add an
ordered `if version < N:` block; tests assert the expected version so accidental drift fails CI.
Only `version < 1` exists today (creates `candidate_segments`).
@backend/infrastructure/database.py::Database._init_db
- § `videos` table ? `id` TEXT PK (file MD5), `filepath`, `processed_at`, `status`,
`validation_results` (JSON text). @backend/infrastructure/database.py (DDL in `_init_db`)
- § `play_segments` table ? confirmed segments; `id` INTEGER PK AUTOINCREMENT, `video_id`
FK?videos `ON DELETE CASCADE`, `start_time`/`end_time`/`duration` REAL, `server`, `receiver`,
`metadata` (JSON text, extensibility point). @backend/infrastructure/database.py (DDL in

```

```

`_init_db`)
- $ `events` table ? `id` INTEGER PK`, `segment_id` FK?play_segments `ON DELETE CASCADE`,
`timestamp`, `type`, `player`, `details` (JSON text). @backend/infrastructure/database.py (DDL
in `_init_db`)
- $ `candidate_segments` table (schema v1) ? raw pre-confirmation detections, detector-agnostic.
Cols: `video_id` FK?videos CASCADE, `detector` TEXT NOT NULL, `chunk_index`,
`start_time`/`end_time`/`duration`, `confidence`, `stats` (JSON), `detection_params` (JSON),
`confirmed_segment_id` FK?play_segments `ON DELETE SET NULL`, `human_label`, `notes`,
`created_at`. ? Plugin extension point: a new segmenter reuses this table by setting its own
`detector`/`stats`/`detection_params` ? common fields are columns, detector-specific metrics go
in `stats` JSON. Index `idx_candidates_video_detector` on `(video_id, detector)`.
@backend/infrastructure/database.py (DDL in `_init_db`)
- `Database.add_video(video_id, filepath, status, validation_results)` ? `INSERT OR REPLACE`
(re-ingest path); JSON-encodes `validation_results`; returns bool. ? `INSERT OR REPLACE` on the
videos PK triggers the CASCADE delete of prior `play_segments`/`events` ? this is the re-ingest-
replaced-prior behavior the report flags.
@backend/infrastructure/database.py::Database.add_video
- `Database.add_play_segment(video_id, start_time, end_time, server=, receiver=, metadata=)` ?
computes `duration = end_time - start_time`; JSON-encodes `metadata`; returns new rowid or None.
@backend/infrastructure/database.py::Database.add_play_segment
- `Database.add_event(segment_id, timestamp, event_type, player=, details=)` ? note param is
`event_type` but the column is `type`; JSON-encodes `details`; returns bool.
@backend/infrastructure/database.py::Database.add_event
- `Database.add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=,
confidence=, stats=, detection_params=)` ? persists one raw detection;
`stats`/`detection_params` JSON-encoded; returns candidate id or None.
@backend/infrastructure/database.py::Database.add_candidate_segment
- `Database.link_candidate_to_segment(candidate_id, segment_id)` ? sets `confirmed_segment_id`
on a candidate (confirmation linkage for the fine-tuning workflow); returns bool.
@backend/infrastructure/database.py::Database.link_candidate_to_segment
- `Database.query_candidate_segments(video_id, detector=None, only_unlabeled=False)` ? dynamic
WHERE; `only_unlabeled` filters `human_label IS NULL`; deserializes `stats`/`detection_params`
from JSON; ordered by `start_time`.
@backend/infrastructure/database.py::Database.query_candidate_segments
- `Database.get_video(video_id)` ? single row; deserializes `validation_results` (default
`""`); returns dict or None. @backend/infrastructure/database.py::Database.get_video
- `Database.query_play_segments(video_id, filters)` ? $ filter contract: `duration_min` (float),
`server` (str), `receiver` (str), `event_type` (str, matched via `EXISTS` subquery on
`events.type`). Hydrates each segment's `metadata` JSON and nested `events` (with `details`
JSON). ? All filter keys use truthiness (`filters.get(...)`) so `duration_min=0`, empty-string
server/receiver are treated as "no filter".
@backend/infrastructure/database.py::Database.query_play_segments
- `Database.clear_video_data(video_id)` ? `DELETE FROM videos WHERE id = ?`; relies on CASCADE
to purge segments/events/candidates.
@backend/infrastructure/database.py::Database.clear_video_data

@backend/reporting/__init__.py
- Module docstring contract: `obsreport` is a SOFT dependency ? absent ? no-op;
`emit_indexer_report` is designed for a `finally` block and NEVER raises. ? Emitting a report
must not break a processing run. @backend/reporting/__init__.py
- `OBSREPORT_AVAILABLE` const ? set True only if `import obsreport` AND `from . import harvest`
both succeed; any exception ? False. ? Soft-dependency gate that toggles the whole reporting
subsystem. @backend/reporting/__init__.py::OBSREPORT_AVAILABLE
- `report_enabled(config)` ? reads `config["report"]["enabled"]`, default True.
@backend/reporting/__init__.py::report_enabled
- `emit_indexer_report(*, db, video_id, video_path, config, val_results, val_context=None,
segmenter_requested=None, segmenter_actual=None, was_reingest=False, segmentation_ran=True)` ?
single entrypoint. Normalizes a typed config (calls `config.model_dump(mode="json")` if
`hasattr(config, "model_dump")`, e.g. `backend.config.AppConfig`) to a plain dict; short-
circuits to None if unavailable/disabled; pulls `play_segments`/`candidates` from `db`;
delegates to `harvest.record_run`; writes via
`rec.write(output_dir=config["report"]["output_dir"])`; returns `res.paths` or None. ? Keyword-
only args; wraps everything in a bare `except Exception: return None` ? failures are swallowed
silently by design. @backend/reporting/__init__.py::emit_indexer_report

```

```

@backend/reporting/harvest.py
- Module docstring contract: pure-ish harvest ? records raw FACTS into an
`obsreport.RunRecorder`; the footgun RULES (signals?flags) live declaratively in spec.yaml, not
here. Imported only after `__init__` confirms obsreport is installed.
@backend/reporting/harvest.py
- ? `_AI_SEGMENTERS` const ? `{"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}`; the
set that classifies a run as AI-based (drives `_ai_stage`). Extension point: add new AI
segmenters here. @backend/reporting/harvest.py::_AI_SEGMENTERS
- `_SPEC_PATH` const ? `spec.yaml` resolved next to this module.
@backend/reporting/harvest.py::_SPEC_PATH
- `load_spec() -> obsreport.ReportSpec` ? `obsreport.load_spec(_SPEC_PATH)`.
@backend/reporting/harvest.py::load_spec
- `video_meta_from_context(val_context, video_path)` ? derives VideoMeta from the validators'
shared `val_context["ffprobe_meta"]` (streams[0] for w/h/`r_frame_rate`/codec, format for
duration/container) + `os.path.getsize` for `size_mb`. $ Sets `fps_source` ? {`unknown`,
`probed`} (later promoted to `fallback`). ? Parses fps from the `"num/den"` `r_frame_rate`
string; guards `ValueError`/`ZeroDivisionError`.
@backend/reporting/harvest.py::video_meta_from_context
- `_validation_stage(rec, val_results)` ? stage `"validation"`; metrics
`checks_passed`/`checks_total`; appends `FAILED <name>: <msg>` per failure; status
`error`/`ok`/`not_run`. ? Reads results via `getattr` duck-typing (`passed`, `validator_name`,
`message`) ? results need not be a fixed type. @backend/reporting/harvest.py::_validation_stage
- `_segmentation_stage(rec, play_segments, candidates, segmenter_requested, segmenter_actual,
config_default, was_reingest, ran)` ? stage `"segmentation"`; metrics `candidate_windows`,
`segment_count` (audience both), `total_play_s` (unit s); reads `detector` tag from
`play_segments[0].metadata`. $ Sets `details`: `segmenter_requested`, `segmenter_actual`,
`config_default`, `segmenter_explicitly_requested`, `matches_config_default`, `was_reingest`,
`detector`, and `metadata_source`. ? `matches_default` only computed when `ran` AND
segmenter_actual AND config_default` (defaults True otherwise) ? an explicit override or halted
run is deliberately NOT a divergence. ? `metadata_source` marks `motion` segmenter output as
"synthetic/heuristic ? NOT measured". @backend/reporting/harvest.py::_segmentation_stage
- `_ai_stage(rec, config, segmenter_actual, play_segments, ran)` ? stage `"ai_handoff"`;
`ai_based = segmenter_actual in _AI_SEGMENTERS`; derives `provider = config["ai_provider"]` or
"gemini", key env `f"{provider.upper()}_API_KEY"`. $ `details`: `ai_based`, `provider`,
`model`, `api_key_present` (`bool(os.environ.get(key_env))`); metric `confirmed_segments` only
when ai_based. ? `api_key_present` is the exact signal `silent_provider_noop` keys on; status
`not_run` (not ran) vs `skipped` (ran but non-AI). @backend/reporting/harvest.py::_ai_stage
- `_highlights(rec, play_segments, video_meta)` ? computes `coverage_pct =
100*total/duration_s`; emits a "No highlights were produced" note when empty; calls
`rec.set_highlights(...)`. @backend/reporting/harvest.py::_highlights
- `record_run(*, video_id, video_path, config, val_results, val_context, play_segments,
candidates, segmenter_requested=None, segmenter_actual=None, was_reingest=False,
segmentation_ran=True, tool_version=None, spec=None, **recorder_kwargs) ->
obsreport.RunRecorder` ? top-level assembler (does NOT write). Builds `RunRecorder("badminton-
highlight-indexer", video_id_kind="file_md5", spec=spec or load_spec(), ...)`; reads
`config_default = config["indexing"]["default_segmenter"]`; runs the 4 stage builders +
`_highlights`; calls `rec.finalize()`. ? fps fallback: if `fps_source == "unknown"` and
`segmentation_ran`, promotes to `"fallback"` and forces `fps = 30.0` (mirrors runners'
`_probe_fps_width` ? 30.0/1280) so the report reads "30 (fallback)" not "? (fallback)".
@backend/reporting/harvest.py::record_run

@backend/reporting/spec.yaml
- $ Report layout/spec consumed by `obsreport` engine (engine = config language + renderers;
this file = indexer-specific layout + footgun rules). Top-level: `name: indexer`, `title`.
@backend/reporting/spec.yaml
- $ `customer_verdict` ? messages for `pass` / `pass_with_warnings` / `fail`.
@backend/reporting/spec.yaml::customer_verdict
- $ `sections` ? ordered list of `{id, title, kind, audience}`: `run` (identity/both), `video`
(video_meta/both), `highlights` (highlights/customer), `cust_notes` (flags/customer), `stages`
(stages/technical), `findings` (flags/technical), `verdict` (verdict/both). ? `kind` selects an
obsreport renderer; `audience` ? {both, customer, technical}.
@backend/reporting/spec.yaml::sections
- ? $ `rules` ? declarative footgun engine: each rule is `{code, severity, when[{path, op,
value}], message, faq_ref, customer_message?}`. `when` clauses are AND'd; `path` dotted-
navigates the recorder JSON (e.g. `stages.ai_handoff.details.api_key_present`,

```

```

`stages.segmentation.metrics.segment_count`, `video_meta.fps_source`). ? These `path` strings
are a hard coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py
silently breaks the matching rule. Extension point: add new rules here, not in code.
@backend/reporting/spec.yaml::rules
- Rule `silent_provider_noop` (error) ? `api_key_present == false` AND `segment_count == 0` ?
missing-credential, not "no rallies"; faq Q7; has `customer_message`.
@backend/reporting/spec.yaml::rules.silent_provider_noop
- Rule `ai_empty_vs_no_rallies` (warn) ? `ai_based == true` AND `api_key_present == true` AND
`segment_count == 0` ? provider error/quota/WSL healthcheck; faq Q7.
@backend/reporting/spec.yaml::rules.ai_empty_vs_no_rallies
- Rule `fps_fallback_used` (warn) ? `video_meta.fps_source == fallback` ? timing/motion
thresholds may be miscalibrated; faq Q14. @backend/reporting/spec.yaml::rules.fps_fallback_used
- Rule `synthetic_motion_metadata` (warn) ? `segmentation.details.segmenter_actual == motion` ?
per-segment metadata are placeholder/unmeasured; faq Q10.
@backend/reporting/spec.yaml::rules.synthetic_motion_metadata
- Rule `segmenter_divergence` (info) ? `matches_config_default == false` AND
`segmenter_explicitly_requested == false` ? a fallback default kicked in; defaults known to
diverge across config.json/setup.py/CLI; faq Q7.
@backend/reporting/spec.yaml::rules.segmenter_divergence
- Rule `reingest_replaced_prior` (info) ? `was_reingest == true` ? re-process replaced prior
segments (keyed by file MD5, cascade delete); faq Q11.
@backend/reporting/spec.yaml::rules.reingest_replaced_prior
- Rule `validation_failed` (error) ? `stages.validation.status == error` ? indexing halted; faq
Q13; has `customer_message`. @backend/reporting/spec.yaml::rules.validation_failed

```

Cross-cutting notes:

```

- ? Coupling chain: harvest.py writes stage/metric/detail names ? spec.yaml
`rules[].when[].path` reads them by dotted string. The pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
- ? The `db` arg to `emit_indexer_report` is the `Database` instance from database.py ?
reporting calls only `query_play_segments(video_id, {})` and
`query_candidate_segments(video_id)` on it (read-only).
- No `tests/` directory was in the supplied path list, so no test-to-subsystem mapping is
included.

```